Lab Sheet-07

Object Oriented Programming Using Java

LastDate:23-10-2026

MCA III Semester

1. Divide by Zero Exception (ArithmeticException**)**

Write a program to accept two integers from the user and perform division.
Handle the case where the denominator is zero using try-catch-finally.

2. Array Index Out of Bounds

Create an array of 5 elements. Ask the user to enter an index and display the value at that index.
Handle invalid index using ArrayIndexOutOfBoundsException.

3. Multiple catch blocks

Write a program to accept two numbers (as strings), convert them to integers, and divide.
Handle:

- NumberFormatException
- ArithmeticException
  using **separate catch blocks**.

4. Nested try blocks

Write a program with a nested try structure:

- Inner try: Access an array element.
- Outer try: Divide two numbers.
  Use separate catch blocks accordingly.

5. finally block demonstration

Write a program that opens a resource (e.g., message "File opened"), performs some operation, and always prints "Resource closed" in a finally block – even if an exception occurs.

6. Using throw keyword

Write a method checkAge(int age) that throws an exception if age < 18.
Call the method and handle the exception in main().

7. Creating User-Defined Exception (Custom)

Create a custom exception InvalidPasswordException.
If the user enters a password with less than 6 characters, throw this exception. Handle it in main().

8. Using throws keyword

Write a method to read a number from the user and convert to integer.
Declare the method using throws NumberFormatException and handle the exception in the calling method.

9. Finally vs Return

Write a method with a try block that returns a value.
Use a finally block to print a message.
Observe and explain which block is executed first.

10. Banking example – User-defined + built-in exceptions

Create a BankAccount class with balance.
Method withdraw(amount):

- If amount > balance, throw InsufficientFundsException (user-defined).
- If amount is negative, throw IllegalArgumentException (built-in).
  Handle both in main().

Intermediate

1. Create and use a simple user-defined package

Create a package named **mypack** containing a class `Hello` with a method `display()`.
In another program, import the package and call the method.

2. Package with multiple classes

Create a package **shapes** containing two classes: `Circle` and `Rectangle`, each having an `area()` method.
Import the package and calculate the area of both shapes in another class.

3. Package with default and public classes

Create a package **testpack**.
Inside it, define one **public class** and one **non-public (default) class**.
Try accessing both from another program. Observe what happens.

4. Subpackages (nested packages)

Create a package **company.employees** with a class `Employee`.
Create another program in package **company.main** that imports and uses the `Employee` class.

5. Access modifiers in packages

Create a package **demo** with a class having `public`, `protected`, `default`, and `private` members.
Write another class (outside the package) and test which members are accessible.

6. Using `import` vs fully qualified name

Create a package **animals** with a class `Dog`.
In another program, call `Dog` class using:
(a) **import animals.Dog;**
(b) **animals.Dog obj = new animals.Dog();**
Compare both methods.

7. Using `import package.*` (wildcard import)

Create multiple classes (e.g., `A`, `B`, `C`) in package **mypackage**.
Use `import mypackage.*;` and access all classes in a single program.

8. Using Java built-in package (java.util)

Write a program that uses `ArrayList` and `Scanner` classes from `java.util` package.
Demonstrate how system packages are imported and used.

9. Create a utility package and reuse it

Create a package **utils** with a class `MathUtils` containing static methods: `add()`, `subtract()`, `multiply()`.
Import the package into another program and call these methods.

10. Packaging and compiling with directory structure (command-line)

Manually create a package **college.students** containing class `Student`.
Compile using `javac -d . Student.java`.
Write a main program in another file to access the class.
Run the program using `java college.students.Main`.

Advance

1. Write a Java program to create a thread by extending the Thread class and display a message five times.
2. Write a Java program to create a thread by implementing the Runnable interface and display numbers from 1 to 10.
3. Explain and demonstrate the life cycle of a thread using a Java program (New, Runnable, Running, Blocked, Terminated).
4. Create two threads that run simultaneously and print their own names. Observe the output and explain thread scheduling.
5. Write a program to set different thread priorities (MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY) and observe which thread executes first.
6. Create a program where one thread sleeps for 2 seconds using Thread.sleep() and another thread continues execution. Explain the behavior.
7. Write a Java program that starts two threads where one thread waits for another to finish using the join() method.
8. Create a program where multiple threads access a shared counter variable. Without synchronization, show incorrect results.
9. Modify the previous program to use synchronized keyword or synchronized method to handle thread synchronization and display correct results.
10. Write a Java program demonstrating a real-life example of multithreading (e.g., downloading files, processing data, or printing tasks) and explain each thread's role.